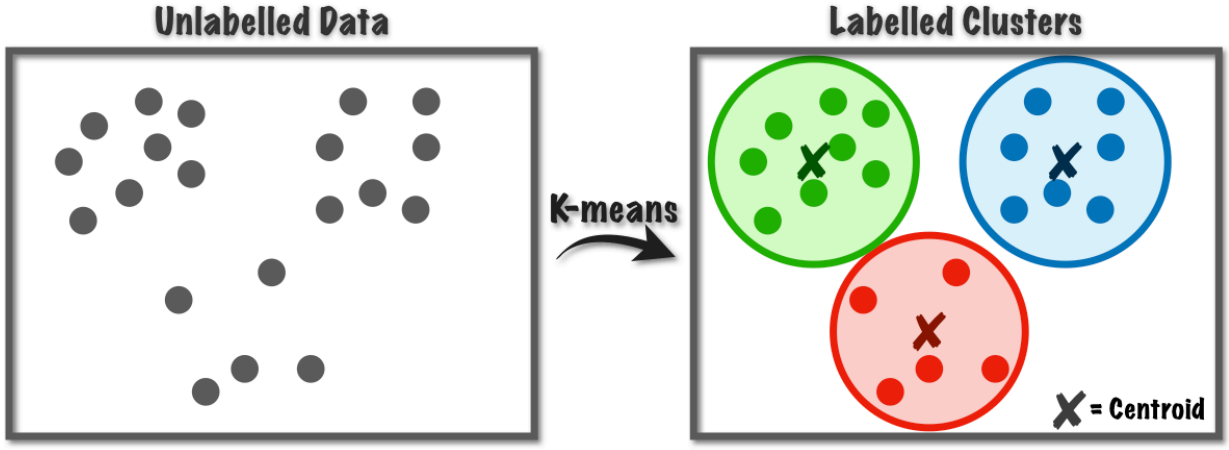
Introduction

K-means clustering is a popular unsupervised(no output column) machine learning algorithm used to group similar data points into clusters|groups. E.g. : (image taken from Medium)



But how many clusters would this algorithm make? That's what we have to decide and tell that algorithm "Make n number clusters in this unlabelled dataset". The optimal way to find how many clusters we can make from a dataset is the **Elbow Method**.

Implementation

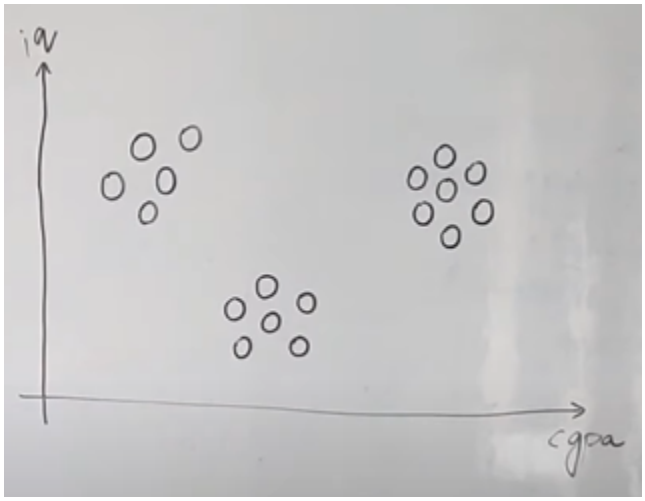
You can watch K-Means Clustering Algorithm which campusx explained very beautifully and I will still write all the info below from that tutorial.

Steps :

1) Decide K (number of clusters we want),
2) Initializing centroids,
3) Assign clusters to each centroid,
4) Move Centroids.
5) Repeat steps 3 and 4 until there is no need to cluster.

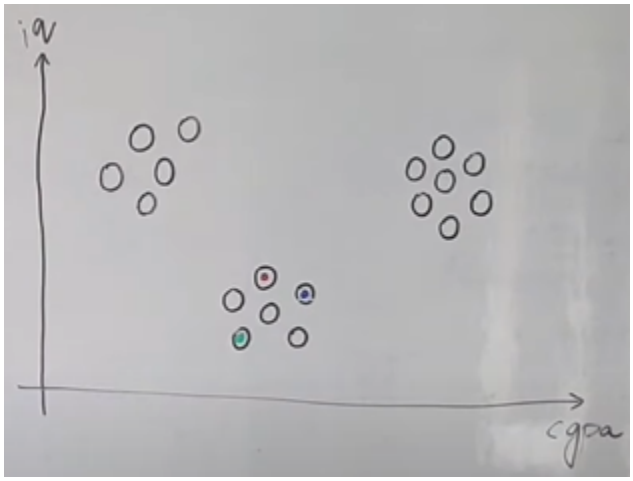
1) Decide K (number of clusters we want) :

Let's assume we want 3 clusters in the below 2D dataset,



2) Initializing centroids :

Since we assumed k = 3. So this algorithm will **randomly** pick any 3 points as centers|centroids of the 3 clusters|groups.

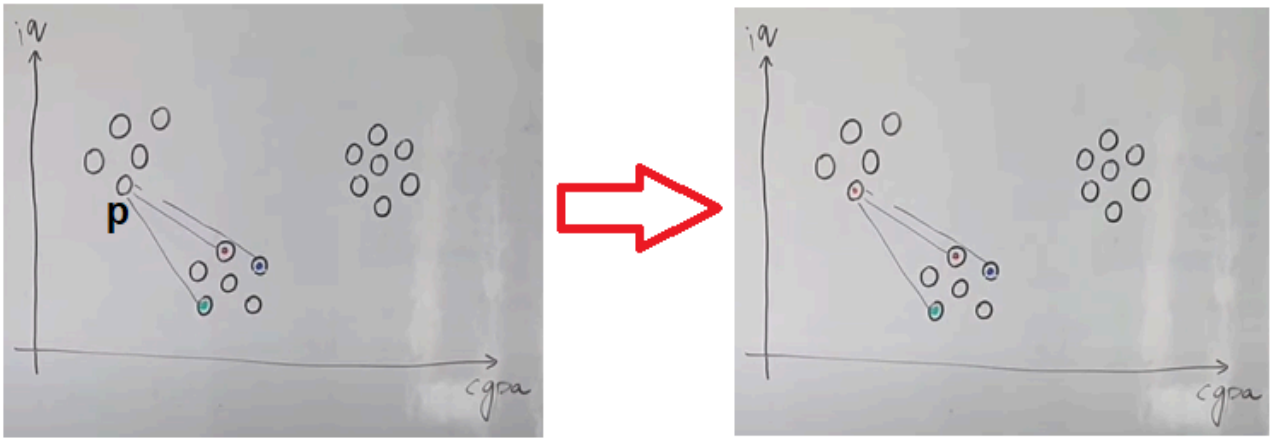


Here the red point is the center of Red Cluster Points that this algorithm will make at the end. Similarly the blue and green point is the center of Blue and Green Cluster Points respectively.

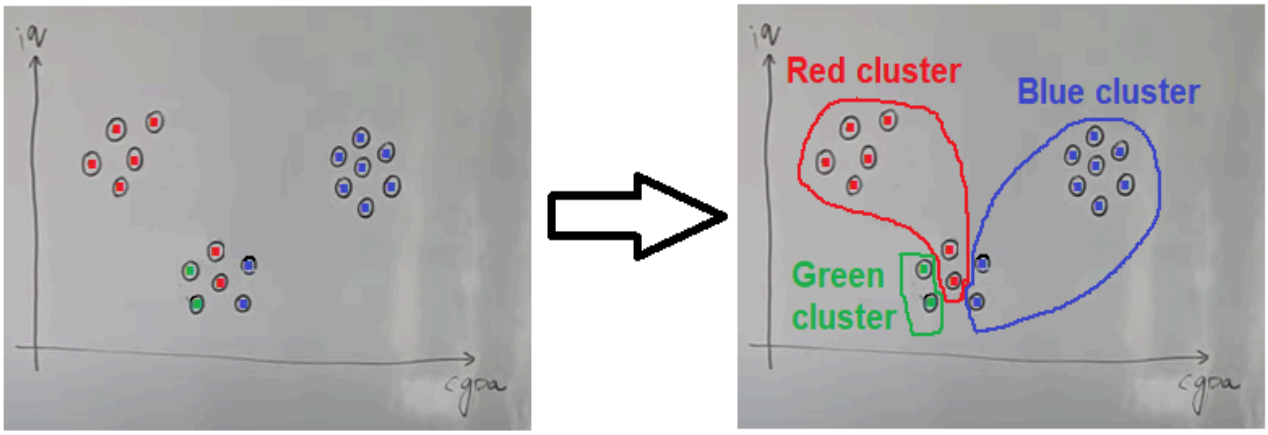
3) Assign clusters to each centroid :

In step 2 we assumed the red, blue and green points are the center of the Red, Blue and Green Clusters respectively but yet we don't know which points belong to which cluster because we haven't assigned the rest of the points to its respective group.

How do we know a point belongs to which group|cluster? Using **Euclidean Distance**.

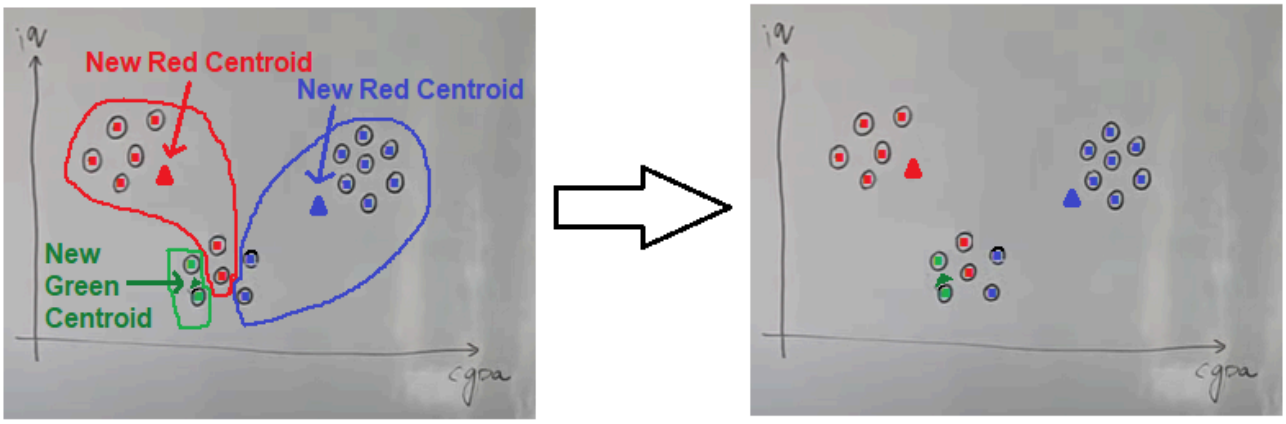


Here from the **p** point we calculated the euclidean distance to each centroid(red, blue, green). The euclidean distance from the point **p** to Red Centroid is lowest, so we assign the point **p** to Red Cluster. Similarly we do with the rest points :



4) Move Centroids :

Now again we've to calculate centroids for each cluster that we just made in the above image :



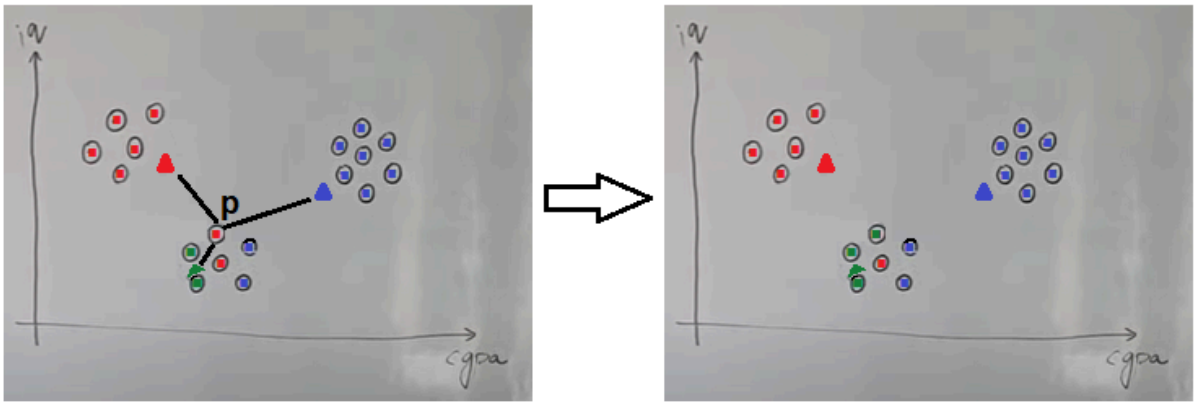
How do we calculate the center of the Red Cluster?

(mean of all the red points) == (mean of cgpa, mean of iq)

Similarly we calculate the center of the Blue and Green Cluster.

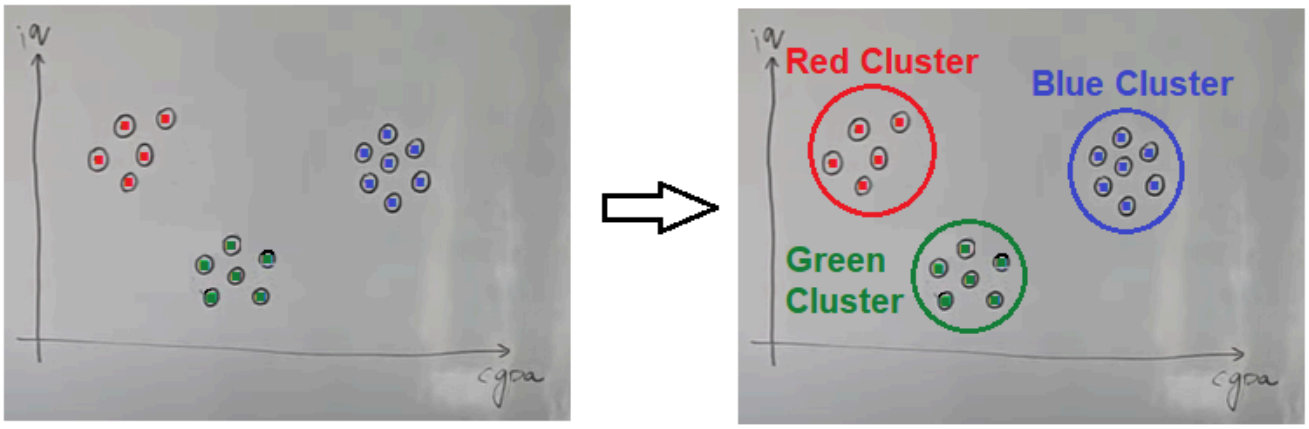
This step is almost the same as step 2 where we've to assign/reassign centroids.

5) Step 3 : Assign clusters to each centroid :



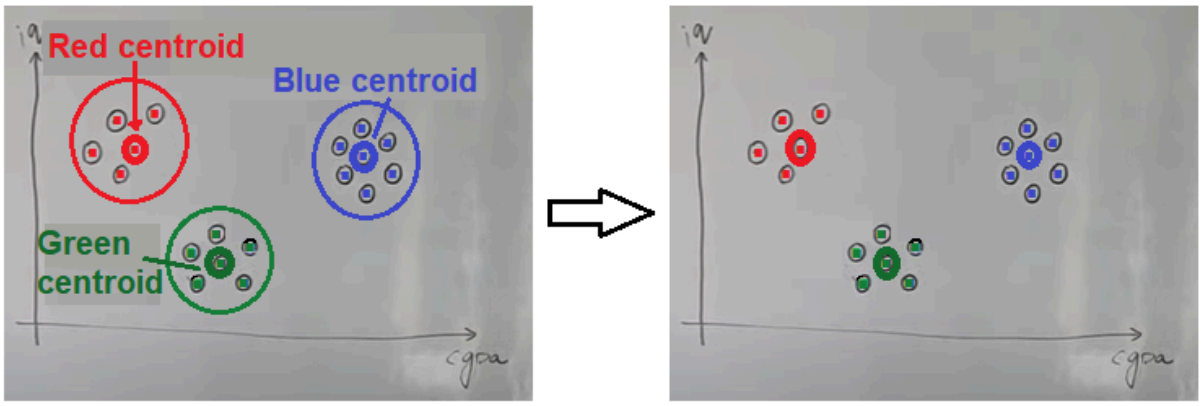
Here from the **p** point we calculated the euclidean distance to each centroid(red, blue, green). The euclidean distance from the point **p** to

Green Centroid is lowest, so we assign the point **p** to Green Cluster.
Similarly we do with the rest points :



6) **Step 4 : Move Centroids :**

Now again we've to calculate centroids for each cluster that we just made in the above image :



How do we calculate the center of the Red Cluster?

(mean of all the red points) == (mean of cgpa, mean of iq)

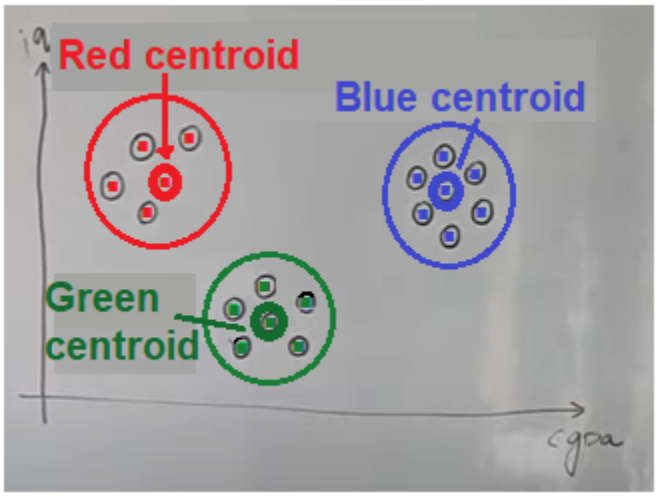
Similarly we calculate the center of the Blue and Green Cluster.

7) **Step 3 : Assign clusters to each centroid :**

We can clearly see there will be no changes in the above right image.

8) **Move Centroids :**

It will be the same as the above left image :



Now it doesn't matter what we do, all the centroid's position won't be changed at all. This is the stopping point. **Since all the previous centroid's position is the same as the new centroid's position, we finish the algorithm**.

Stopping Point

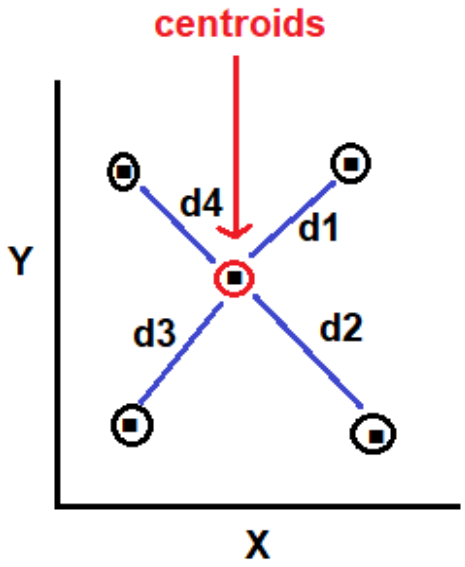
So we keep making centroids and assigning the points to their corresponding cluster until all the previous centroid's position is the same as the new centroid's position.

How to calculate how many optimal clusters we need from a dataset?

Using the Elbow method and Elbow Point. A crucial calculation, WCSS, is needed to know first.

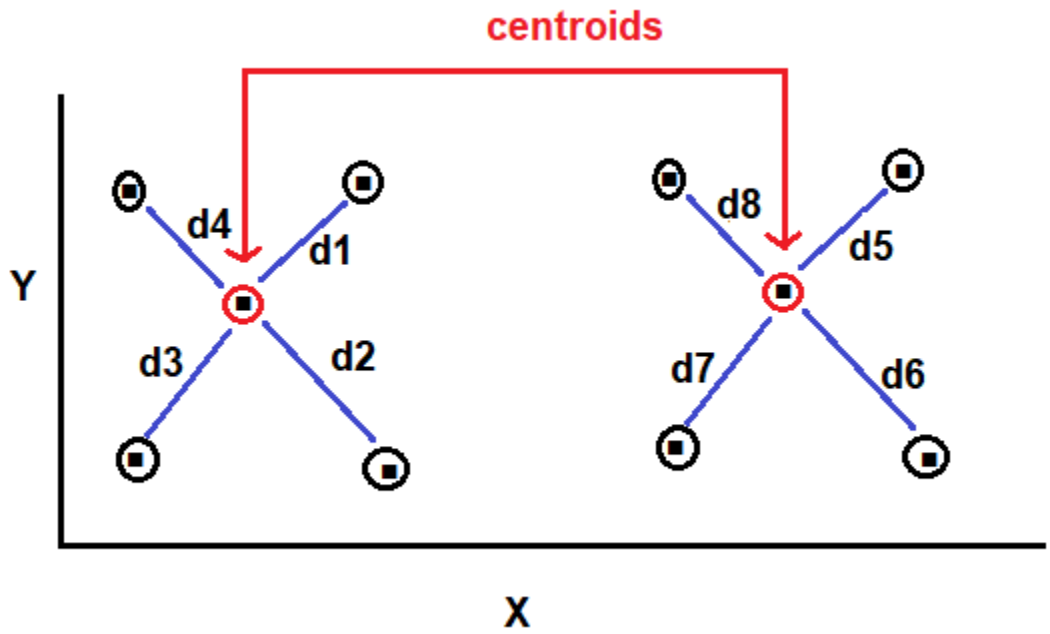
WCSS (Within-Cluster Sum of Squares) :

WCSS for dataset with 1 cluster :



WCSS $= d1^2 + d2^2 + d3^2 + d4^2$

WCSS for dataset with 2 clusters :

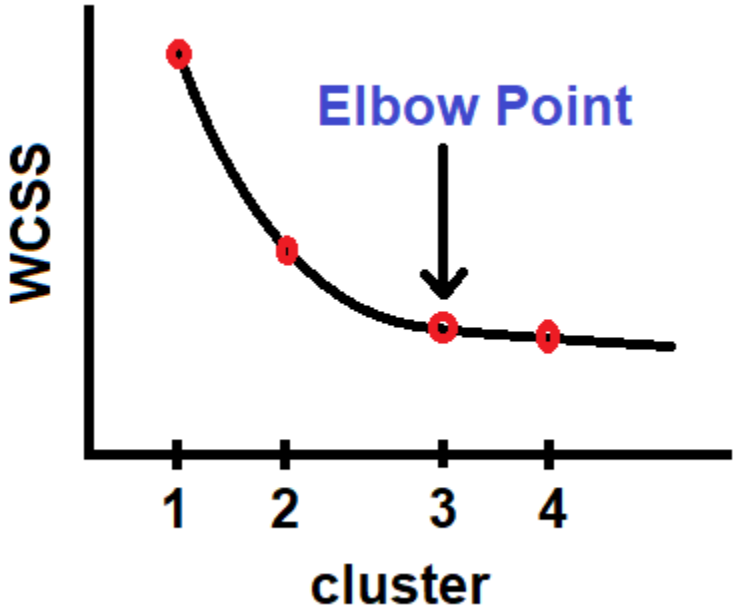


$$WCSS = WCSS_{1st\ cluster} + WCSS_{2nd\ cluster}$$

$WCSS_{1st\ cluster} = d1^2 + d2^2 + d3^2 + d4^2$

$WCSS_{2nd\ cluster} = d5^2 + d6^2 + d7^2 + d8^2$

In this way we calculate WCSS for n clusters. The elbow method is :

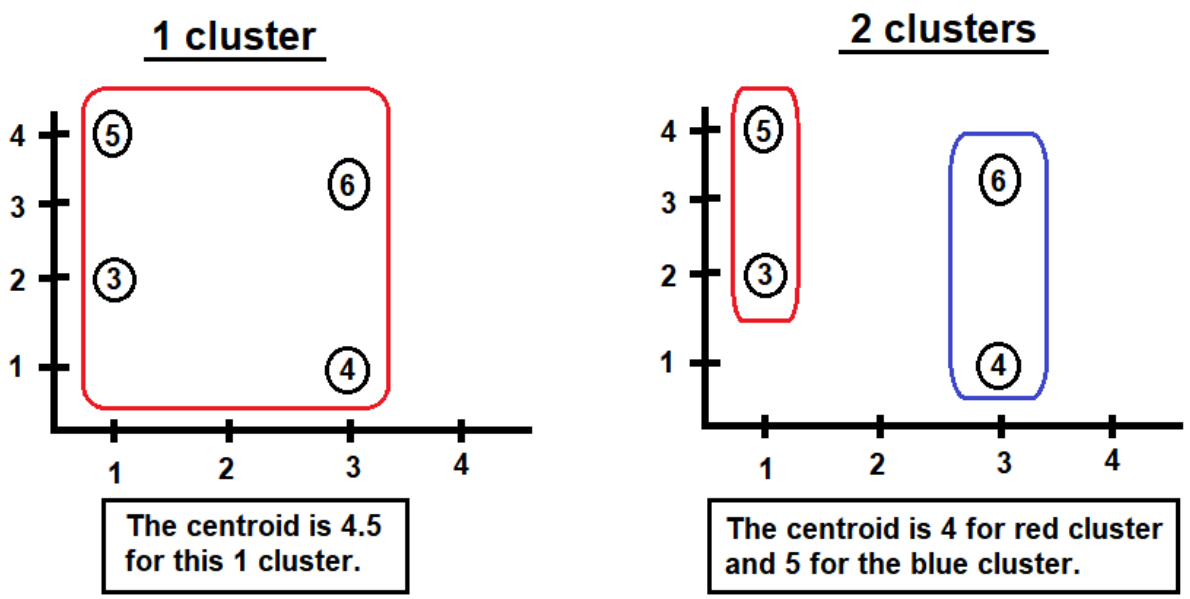


Note : **WCSS is also called Inertia.**

As you can see from 1 cluster to 2 clusters, there is significant change. But between 3 clusters and 4 clusters, there is very few changes denoting clustering|grouping more than 3 won't bring any significant variation inside the extra(4th, 5th.. clusters) clusters|groups and obviously we want to cluster those points which are unique than the others. So, 3 is Elbow Point here denoting the optimal clustering number is 3.

How 1 cluster has the bigger WCSS and the 4 clusters has lower WCSS?

For example :



For 1 cluster => WCSS = $(5 - 4.5)^2 + (3 - 4.5)^2 + (4 - 4.5)^2 + (6 - 4.5)^2 = 5$.

For 2 clusters => WCSS = $WCSS_{1st\ cluster} + WCSS_{2nd\ cluster}$

$= (5 - 4)^2 + (3 - 4)^2 + (4 - 5)^2 + (6 - 5)^2 = 4$.

Of course we are not seeing drastic changes between 1 cluster and 2 clusters because I used single points where each point is made of 2 values.

More simply : If you have 4 points and you make 4 clusters i.e. each point is a cluster. Then for each cluster the centroid is that value itself and distance from a value to itself is 0. So ultimately we'll have WCSS = 0 for 4 points. But for 1 cluster of all the 4 points, WCSS is positive and highest.

The main equation we used in K Means Algorithm is Euclidean Distance which also works for any ND Data. So everything we did above on 2D Data is same on any ND Data.

Sklearn’s K Means Parameters

---

1) K Means Clustering is an Unsupervised Machine Learning Algorithm, meaning it doesn’t have any output label|column. If it doesn’t have an **actual output label|column**, then we can’t quantify accuracy, r2 score, RMSE etc as they need ‘y_true’ which is the actual output column, not what the model predicts which is ‘y_pred’. We can train and predict just like other models.

2) Each KMeans() has WCSS | inertia parameter. And we saw in “How to calculate how many optimal clusters we need from a dataset” section that we need to calculate the inertia for cluster = 1, 2, 3 … to check which has the inertia after what no more significant changes in y axis(WCSS | Inertia) came and we took the corresponding cluster no. So we have to run multiple KMeans algorithms to calculate the optimal **cluster_no** to set it in the final KMeans algorithm.